PROVISIONAL PATENT APPLICATION

UNIFIED ADAPTIVE COMPUTATIONAL FRAMEWORK WITH MULTI-DOMAIN IMPLEMENTATION EMBODIMENTS

COVER SHEET

Application Type: Provisional Patent Application
Filing Date: December 15, 2025
Applicant: Rogue Intelligence Inc. (United States)
Inventor: MA ZHUOLI
Correspondence Address: [To be completed]
Title of Invention: Unified Adaptive Computational Framework with Multi-Domain Implementation Embodiments

NOTICE: NON-LIMITING DISCLOSURE

This provisional patent application is provided for the purpose of establishing a priority date. All examples, embodiments, performance characteristics, and implementation details described herein are illustrative and non-limiting. The scope of protection sought encompasses all variations and equivalents that fall within the spirit of the disclosed invention.

CROSS-REFERENCE TO RELATED APPLICATIONS

[0001] This application claims priority benefit and is related to:

- Provisional Patent Application: "Unified Weighted Signal Coordination Method for Self-Adaptive Computational Systems" (Filed December 12, 2025)

TECHNICAL FIELD

[0002] The present disclosure relates to adaptive computational frameworks, and more particularly to:
(a) unified deterministic coordination methods for multi-signal processing;
(b) multi-force computational architectures inspired by physical law analogies;
(c) programming language implementations incorporating intelligent coordination;
(d) operating system architectures with adaptive resource management; and
(e) simulation platforms for modeling complex physical systems.

BACKGROUND OF THE INVENTION

[0003] Modern computational systems face challenges in coordinating diverse subsystems, optimizing resource allocation, and adapting to heterogeneous workloads. Existing approaches typically rely on:

- Probabilistic models requiring extensive training data

- Heuristic thresholds requiring manual tuning
- Static configurations lacking adaptability
- Isolated subsystems without unified coordination

[0004] There exists a need for a unified computational framework that:

- Operates deterministically without probabilistic inference
- Provides mathematically bounded, explainable behavior
- Adapts to diverse computational domains
- Supports multiple implementation embodiments across software and hardware platforms

---

HIGH-LEVEL SYSTEM OVERVIEW

[0005] At the highest abstraction level, the disclosed framework operates as a **closed-loop coordination system**. Heterogeneous computational subsystems continuously emit state signals characterized by intensity, direction, and quality attributes. A deterministic coordination engine receives these signals and computes bounded affinity vectors through unified mathematical operations. The resulting affinity values guide system-level behavior, including workload routing, resource allocation, and adaptive optimization. System behavior emerges through iterative feedback cycles, wherein coordination outputs influence subsequent signal generation, enabling continuous self-adaptation without probabilistic training or heuristic parameter tuning.

[0006] This closed-loop paradigm applies uniformly across all implementation embodiments disclosed herein, including programming languages, operating systems, and simulation platforms.

SUMMARY OF THE INVENTION

[0007] Disclosed is a unified adaptive computational framework comprising:

1. **Core Coordination Mechanism**: A deterministic signal aggregation method using unified mathematical formulations for weight calculation and affinity computation.
2. **Multi-Force Architecture**: A quad-system computational architecture where distinct subsystems specialize in complementary computational paradigms, coordinated by a central decision engine.
3. **Programming Language Embodiment**: An intelligent programming language incorporating the coordination mechanism as a runtime foundation layer.
4. **Operating System Embodiment**: An adaptive operating system architecture utilizing the coordination mechanism for resource management, scheduling, and device abstraction.
5. **Simulation Platform Embodiment**: A physical law simulation framework mapping computational forces to fundamental physical interactions for scientific modeling applications.

DETAILED DESCRIPTION OF THE INVENTION

PART I: CORE COORDINATION MECHANISM

Section 1.1: Signal Representation

[0008] The framework processes heterogeneous signals from multiple sources. Each signal is represented as a structured tuple comprising:

- **Intensity (I)**: A real-valued scalar representing signal magnitude
- **Direction (D)**: An n-dimensional vector indicating directional influence across coordination dimensions
- **Quality Score (Q)**: A scalar representing reliability, trust, or priority

[0009] Signals may originate from heterogeneous subsystems and are not required to share a common semantic meaning or source.

Section 1.2: Unified Weight Computation

[0010] For each signal s, a deterministic weight w is computed using the formulation:

```
w = I × Q
```

[0011] This formulation ensures that signal influence is jointly determined by magnitude and quality without reliance on heuristic thresholds or probabilistic parameters.

Section 1.3: Affinity Aggregation

[0012] For each coordinate dimension k of the coordination space, an affinity value A[k] is computed as:

```
A[k] = Σ(D[k] × w) / Σ(w)
```

[0013] The aggregation is performed independently per dimension, enabling coordination across arbitrary-dimensional state spaces while maintaining computational efficiency.

Section 1.4: Pseudocode (Illustrative)

[0014] The following pseudocode illustrates one non-limiting implementation:

```
function COORDINATE(signals):
    total_weight = 0
    affinity = zero_vector()

    for each signal s in signals:
        w = s.intensity * s.quality
        total_weight += w
        affinity += s.direction * w

    if total_weight ≠ 0:
        affinity = affinity / total_weight

    return affinity
```

[0015] The pseudocode is provided for explanatory purposes only and does not restrict the invention to any particular programming language, data structure, or execution environment.

Section 1.5: Mathematical Properties (Non-Limiting)

[0016] Under defined operating constraints, the coordination mechanism exhibits the following properties:

1. **Deterministic Behavior**: Identical inputs produce identical outputs through explicit mathematical operations.
2. **Bounded Aggregation**: Bounded inputs yield bounded affinity values via weighted averaging.
3. **Quality-Weighted Influence**: Signals with higher quality scores contribute proportionally more to aggregated outcomes.
4. **Dimensional Independence**: Aggregation is computed independently per dimension, enabling operation in arbitrary-dimensional spaces.
5. **Cyclic Feedback Compatibility**: Outputs may be used to update signal parameters for subsequent coordination cycles.

PART II: MULTI-FORCE COMPUTATIONAL ARCHITECTURE

Section 2.1: Quad-System Overview

[0017] The framework includes a multi-force computational architecture comprising four distinct subsystems, each specializing in complementary computational paradigms:

Force System	Computational Specialization	Physical Analogy
System A	Aggregation, compression, high-density data processing	Gravitational attraction
System B	Network connectivity, distributed processing, pattern correlation	Electromagnetic interaction
System C	Parallel computation, energy distribution, phase transitions	Strong nuclear cohesion
System D	Fundamental signal processing, enhancement, feedback optimization	Weak nuclear signaling

[0018] The physical analogies are provided for conceptual illustration only and do not limit the computational implementation.

Section 2.2: Central Decision Engine

[0019] A central decision engine coordinates the quad-system architecture by:

1. Receiving signal inputs from all subsystems
2. Computing unified weights and affinities using the core coordination mechanism
3. Generating system preference vectors indicating relative activation levels
4. Distributing computational workloads based on preference values

[0020] Illustrative system preference calculation:

```
preferences = {
    system_a_affinity: weighted_direction[0] / total_weight,
    system_b_affinity: weighted_direction[1] / total_weight,
    system_c_affinity: weighted_direction[2] / total_weight,
    system_d_affinity: weighted_direction[3] / total_weight,
}
```

Section 2.3: Subsystem Signal Generation

[0021] Each subsystem generates deformation signals reflecting its computational state:

```
signal = {
    intensity: computational_load_metric,
    direction: [a, b, c, d],  // normalized preference vector
    quality_score: result_confidence_metric,
    timestamp: current_time,
    source_system: system_identifier
}
```

[0022] The direction vector encodes the subsystem's assessment of optimal workload distribution across the quad-system architecture.

Section 2.4: Coordinated Computation Flow

[0023] Algorithm flow for coordinated multi-force computation:

```
Step 1: Receive computation request with workload characteristics
Step 2: Collect current state signals from all four subsystems
Step 3: Apply core coordination mechanism to aggregate signals
Step 4: Generate system preference vector from affinity values
Step 5: Route workload to subsystem(s) with highest affinity
Step 6: Execute computation within selected subsystem(s)
Step 7: Collect results and generate feedback signal
Step 8: Update system states for subsequent cycles
```

PART III: PROGRAMMING LANGUAGE EMBODIMENT

Section 3.1: Intelligent Language Architecture

[0024] One embodiment of the framework comprises a programming language with the coordination mechanism integrated as a runtime foundation layer:

```
Architecture Stack:
┌─────────────────────────────────────────┐
│         User Application Code           │
├─────────────────────────────────────────┤
│         Syntax Parser/Compiler          │
├─────────────────────────────────────────┤
```

```
│    Coordination Engine Foundation        │ ← Core Mechanism
│    ├── Automatic Resource Management     │
│    ├── Intelligent Memory Control        │
│    ├── Adaptive Dimension System         │
│    └── Performance Optimization          │
├──────────────────────────────────────────┤
│         System Calls / Machine Code      │
└──────────────────────────────────────────┘
```

Section 3.2: Zero-Configuration Programming

[0025] The language embodiment enables zero-configuration programming where:

1. The coordination engine automatically manages infrastructure decisions
2. Resource allocation adapts to program behavior without explicit configuration
3. Performance optimization occurs transparently during execution
4. Users focus on application logic while the system handles optimization

[0026] Illustrative program structure:

```
main() {
    // User writes only business logic
    process_data(input_dataset)

    // Coordination engine automatically:
    // - Selects optimal subsystem for workload
    // - Manages memory allocation
    // - Optimizes execution path
    // - Monitors and adjusts performance
}
```

Section 3.3: Compiler Integration

[0027] The coordination mechanism integrates with compilation as follows:

1. **Analysis Phase**: Characterize code complexity and resource requirements
2. **Optimization Phase**: Apply coordination-guided optimization strategies
3. **Code Generation**: Emit code with embedded coordination hooks
4. **Runtime Phase**: Execute with continuous coordination feedback

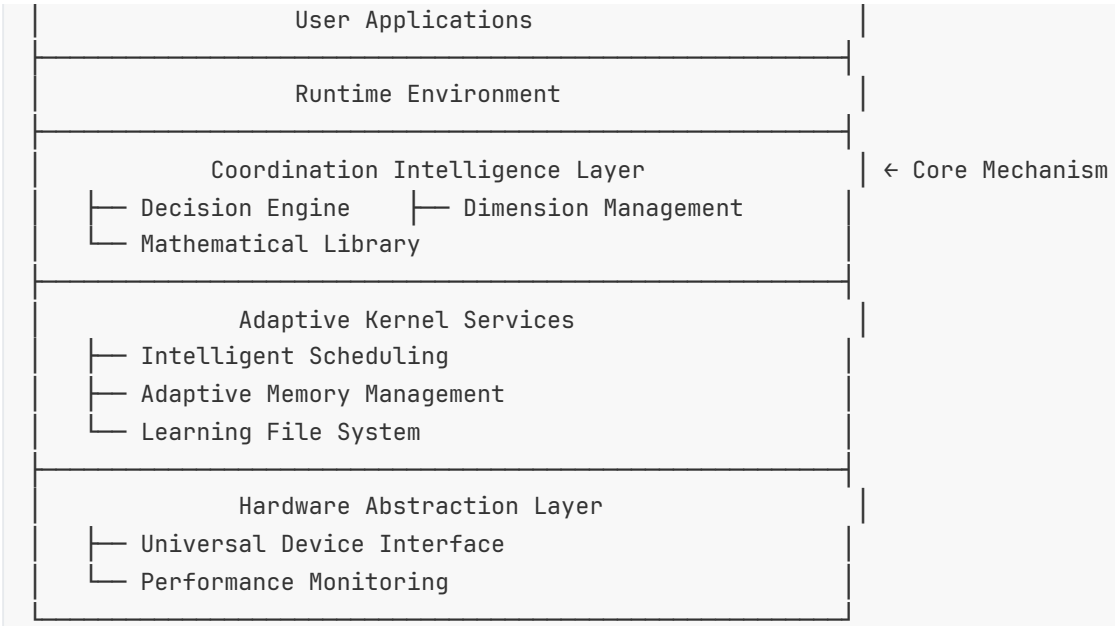
---

PART IV: OPERATING SYSTEM EMBODIMENT

Section 4.1: Adaptive Operating System Architecture

[0028] Another embodiment comprises an operating system architecture utilizing the coordination mechanism:

```
Operating System Architecture:
┌──────────────────────────────────────────────────────────┐
```



Section 4.2: Intelligent Process Scheduling

[0029] The coordination mechanism enables intelligent process scheduling:

```
function schedule_next_process(scheduler):
    // 1. Collect system state signals
    state = collect_system_state()

    // 2. Analyze process requirements
    analysis = analyze_process_requirements(active_processes)

    // 3. Apply coordination mechanism for scheduling decision
    decision = coordinate_scheduling_signals(state, analysis)

    // 4. Select optimal process based on affinity values
    return select_highest_affinity_process(decision)
```

Section 4.3: Adaptive Device Management

[0030] Device management without traditional driver models:

1. **Hardware Discovery**: Automatic detection and fingerprinting of devices
2. **Capability Analysis**: Determination of device capabilities through probing
3. **Rule Generation**: Dynamic generation of device interaction rules
4. **Performance Optimization**: Continuous optimization based on usage patterns

[0031] The coordination mechanism guides device-to-subsystem mapping for optimal I/O routing.

Section 4.4: Intelligent Memory Management

[0032] Memory management utilizing coordination intelligence:

```
function smart_allocate(request):
    // 1. Analyze allocation pattern
    pattern = analyze_allocation_pattern(request)

    // 2. Predict memory usage behavior
    prediction = predict_usage_behavior(request, pattern)

    // 3. Coordinate allocation strategy
    strategy = coordinate_allocation_signals(request, prediction)

    // 4. Execute optimal allocation
    return execute_allocation(strategy)
```

PART V: SIMULATION PLATFORM EMBODIMENT

Section 5.1: Physical Law Simulation Framework

[0033] Another embodiment comprises a simulation platform mapping computational forces to fundamental physical interactions:

Computational Force	Physical Analog	Simulation Application
System A	Gravitational Force	Spacetime curvature, black hole physics
System B	Electromagnetic Force	Field interactions, wave propagation
System C	Strong Nuclear Force	Particle binding, fusion processes
System D	Weak Nuclear Force	Decay processes, signal transmission

[0034] The coordination engine serves as a "unified field theory" for the simulation, governing interactions between simulated physical forces.

Section 5.2: Universe Simulation Interface

[0035] Illustrative simulation interface:

```
interface UniverseSimulator {
    // Initialize simulation with physical constants
    function initialize(constants: UniversalConstants)

    // Evolve simulation state over time steps
    function evolve(time_steps: integer) → SimulationState

    // Query simulation for observables
    function observe(query: ObservationQuery) → Observation

    // Coordination engine manages force interactions
    internal coordination_engine: CoordinationMechanism
}
```

Section 5.3: Physical Constants Integration

[0036] The simulation platform may incorporate physical constants:

```
UniversalConstants = {
    speed_of_light: configurable,
    gravitational_constant: configurable,
    planck_constant: configurable,
    fine_structure_constant: configurable,
    // Additional constants as needed
}
```

[0037] The coordination mechanism governs how these constants influence multi-force interactions during simulation evolution.

Section 5.4: Research Applications

[0038] The simulation embodiment enables:

1. **Hypothesis Testing**: Evaluation of physical theories through simulation
2. **Extreme Condition Modeling**: Exploration of conditions impractical for physical experimentation
3. **Multi-Scale Phenomena**: Simulation spanning quantum to cosmological scales
4. **Discovery Assistance**: Identification of novel patterns and relationships

---

PART VI: ADVANCED SYSTEM COMPONENTS

Section 6.1: Dimensional Management System

[0039] The framework includes a dimensional management system supporting:

1. **Adaptive Dimensionality**: Dynamic adjustment of coordinate space dimensions based on computational complexity
2. **Non-Integer Dimensions**: Support for fractional dimensional values (e.g., 2.3, 3.7) for enhanced modeling flexibility
3. **Dimensional Expansion**: Automatic expansion when complexity exceeds current dimensional capacity
4. **Dimensional Compression**: Reduction when stable operation permits simplified representation

Section 6.2: Stability Detection and Optimization

[0040] The framework includes mechanisms for:

1. **Stability Detection**: Monitoring system state variance to detect stable operating regions
2. **Collapse Optimization**: Resource consolidation when system reaches stable state
3. **Snapshot Execution**: Incremental execution avoiding redundant computation during stable periods
4. **Automatic Recovery**: State restoration when stability conditions change

Section 6.3: Bio-Inspired Cellular Networks

[0041] One advanced embodiment comprises a cellular network architecture where:

1. Individual computational cells maintain position preferences mapped to subsystems
2. Cells form connections based on position compatibility
3. Cell evolution is guided by coordination engine feedback
4. Network topology adapts to workload characteristics

[0042] Position types (illustrative):

Position Type	Subsystem Mapping	Behavioral Tendency
Structure	System A	Stability, compression
Transform	System C	Mutation, innovation
Relation	System B	Connectivity, memory
Explore	Mixed	Unknown territory expansion

Section 6.4: External Module Injection and Extension

[0043] The framework further supports **external module injection**, wherein third-party computational subsystems may participate in the coordination mechanism as signal-emitting entities without modification of the core coordination engine. External modules comply with the signal interface specification (intensity, direction, quality score) and are treated equivalently to native subsystems during affinity aggregation.

[0044] This extensibility enables:

1. **Third-Party Integration**: External computational modules may join the coordination ecosystem
2. **Platform Ecosystem Development**: SDK and API interfaces for external developers
3. **Domain-Specific Extensions**: Specialized subsystems for particular application domains
4. **Backward Compatibility**: New modules integrate without altering existing system behavior

[0045] The coordination engine remains agnostic to signal origin, treating all compliant signals uniformly regardless of whether they originate from core subsystems or externally injected modules.

PART VII: IMPLEMENTATION CONSIDERATIONS

Section 7.1: Hardware Implementation

[0046] The framework may be implemented in:

- General-purpose processors with software implementation
- Specialized hardware accelerators
- Field-programmable gate arrays (FPGAs)
- Application-specific integrated circuits (ASICs)
- Distributed computing clusters

- Hybrid hardware/software configurations

Section 7.2: Software Implementation

[0047] Software implementations may utilize:

- Any programming language (illustrative: Rust, C, C++, Python)
- Any data structure for signal representation
- Any execution environment (bare metal, virtual machine, container)
- Any operating system platform

[0048] Code examples provided herein are illustrative only and do not limit scope.

Section 7.3: Scalability

[0049] The framework scales from:

- Embedded systems with constrained resources
- Desktop and workstation applications
- Server and cloud deployments
- Supercomputing installations
- Distributed global networks

CLAIMS

Core Coordination Method Claims

1. A method for deterministic coordination of heterogeneous computational signals, comprising:
    - receiving a plurality of input signals, each signal comprising an intensity value, a direction vector, and a quality score;
    - computing a weight for each signal by multiplying intensity and quality score;
    - aggregating weighted directional components independently for each dimension of a coordination space;
    - normalizing aggregated values by total weight to produce bounded affinity values; and
    - outputting an affinity vector for use in adaptive system behavior.
2. The method of claim 1, wherein the coordination operates deterministically such that identical inputs produce identical outputs.
3. The method of claim 1, further comprising iteratively updating signal parameters based on coordination outputs for cyclic adaptive evolution.
4. The method of claim 1, wherein the direction vector has arbitrary dimensionality.

Multi-Force Architecture Claims

5. A computational system comprising:
    - a plurality of distinct computational subsystems, each specializing in a complementary computational paradigm;
    - a central decision engine implementing the coordination method of claim 1;
    - signal communication pathways between subsystems and the decision engine; and
    - workload routing mechanisms guided by computed affinity values.
6. The system of claim 5, comprising four computational subsystems corresponding to aggregation, network connectivity, parallel computation, and fundamental signal processing paradigms.
7. The system of claim 5, wherein each subsystem generates deformation signals reflecting computational state for processing by the decision engine.

Programming Language Claims

8. A programming language implementation comprising:
    - a syntax parser for processing user application code;
    - a runtime foundation layer implementing the coordination method of claim 1;
    - automatic resource management guided by coordination outputs; and
    - transparent performance optimization without explicit user configuration.
9. The implementation of claim 8, wherein the coordination layer automatically selects optimal computational subsystems for user workloads.
10. The implementation of claim 8, further comprising compiler integration for coordination-guided code optimization.

Operating System Claims

11. An operating system architecture comprising:
    - a coordination intelligence layer implementing the coordination method of claim 1;
    - an adaptive kernel utilizing coordination outputs for process scheduling;
    - intelligent memory management guided by coordination affinities; and
    - device abstraction utilizing coordination for optimal I/O routing.
12. The architecture of claim 11, wherein process scheduling decisions are derived from coordination of system state signals and process requirement analysis.
13. The architecture of claim 11, further comprising adaptive device management without traditional driver models.

Simulation Platform Claims

14. A simulation platform comprising:
    - a multi-force computational architecture where forces correspond to physical interaction analogs;
    - a coordination engine serving as unified interaction governance;

- configurable physical constants influencing force interactions; and
- simulation evolution governed by coordinated force dynamics.

15. The platform of claim 14, wherein four computational forces correspond to gravitational, electromagnetic, strong nuclear, and weak nuclear interaction analogs.
16. The platform of claim 14, enabling simulation of phenomena across quantum to cosmological scales.

Advanced Feature Claims

17. The system of any preceding claim, further comprising a dimensional management system supporting non-integer dimensional values.
18. The system of any preceding claim, further comprising stability detection and collapse optimization mechanisms.
19. The system of any preceding claim, further comprising bio-inspired cellular network architecture with position-based subsystem mapping.
20. A computer-readable medium storing instructions that, when executed, implement the method of claim 1.

ABSTRACT

A unified adaptive computational framework providing deterministic coordination of heterogeneous signals through weight computation (w = I × Q) and affinity aggregation (A[k] = Σ(D[k] × w) / Σ(w)). The framework encompasses a multi-force computational architecture with four specialized subsystems coordinated by a central decision engine. Implementation embodiments include: (1) an intelligent programming language with coordination-based runtime foundation; (2) an adaptive operating system with coordination-guided resource management; and (3) a physical law simulation platform mapping computational forces to fundamental interactions. The framework operates deterministically without probabilistic training, supports arbitrary-dimensional coordination spaces, and enables cyclic adaptive evolution through feedback mechanisms.

FIGURES

FIG. 1: Core Coordination Mechanism Architecture
FIG. 2: Multi-Force Quad-System Architecture
FIG. 3: Programming Language Embodiment Stack
FIG. 4: Operating System Embodiment Architecture
FIG. 5: Simulation Platform Embodiment Structure
FIG. 6: Dimensional Management System
FIG. 7: Coordinated Computation Flow
FIG. 8: Bio-Inspired Cellular Network Architecture

FILING CHECKLIST

- [ ] Cover Sheet (Completed)
- [ ] Specification (This document)
- [ ] Claims (20 claims included)

- [ ] Abstract (<150 words)
- [ ] Figures (8 figures referenced)
- [ ] Filing Fee
- [ ] Inventor Declaration

Document Version: 1.0
Prepared: December 15, 2025
Status: READY FOR FILING

END OF PROVISIONAL PATENT APPLICATION